

Implementation Plan

Roadmap, sprints, task backlog, prompt order, deployment runbook and pilot plan

From the current state to a paid launch: what is already done, what remains, in what order to build it with an AI IDE, how to test and deploy on the Fedora server, how to run the pilot, and what to do in the next seven days.

Version 1.0 · 8 October 2026

Prepared for: Product owner, Gymholik

Status: Working draft for build and pilot

Confidential. Do not share without permission.

Contents

- 1. Approach 3
- 2. Current Status Snapshot 3
- 3. Roadmap and Milestones 4
- 4. Sprint Plan and Task Backlog 4
- 5. AI IDE Prompt Order 7
- 6. Environments and Configuration 8
- 7. Engineering Practices 8
- 8. Test Plan and Release Gates 9
- 9. Deployment Runbook (Fedora Server) 10
- 10. Pilot Plan 10
- 11. Subscription Operations (Phase 1, manual) 11
- 12. Budget Estimate (approximate, verify before paying) 11
- 13. Risk Register (Delivery) 12
- 14. Phase 2 and Phase 3 Backlog 12
- 15. Next Seven Days (Action List) 13

1. Approach

- **Team:** one founder-developer working with AI coding tools (Cursor, Claude Code, Antigravity or Copilot). Plans assume about 25 to 30 productive hours per week.
- **Method:** Agile in one-week sprints. Each sprint ends with a working build on a phone or simulator, a green test run and a git tag.
- **AI IDE rule:** one module per prompt, run it, test it, review it, commit it. Never ask for two modules at once.
- **Quality gate:** mvn clean test must be green (including tenant isolation tests) before any new module or deployment.
- **Order of risk:** security and data correctness first, then daily-use speed, then polish, then deployment, then pilot.

Dates below are estimates starting the week of 12 October 2026. They assume the current code base is as described in section 2. Re-plan after Sprint 1 when the real state is confirmed.

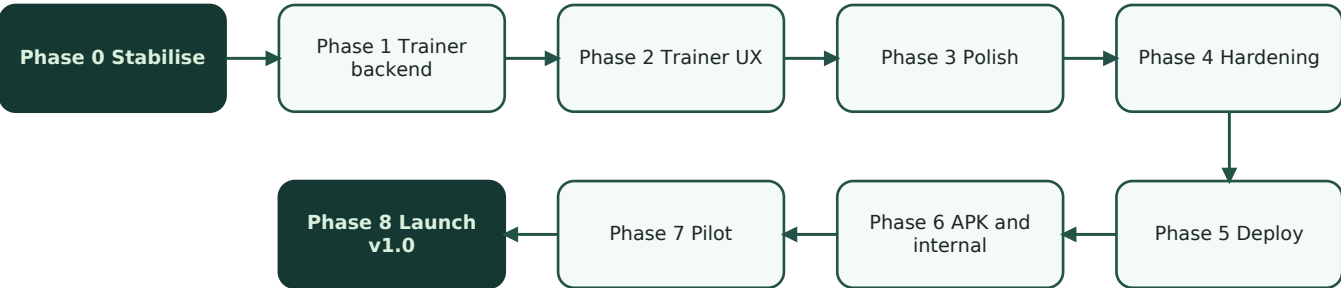
2. Current Status Snapshot

This snapshot is built from the project conversation and screenshots up to 9 October 2026. It is not a code audit; confirm each line in the repository.

Area	Status	Evidence and comment
Product planning	Done	MVP scope, roles, pricing idea, architecture decided (documents 01 to 06)
Backend skeleton and entities	Mostly done	Spring Boot project in gymholik/backend with Dockerfile, docker-compose, Swagger and curl examples
Auth, members, plans, payments, attendance, dashboard	Built, needs verification	Service layer was re-aligned to the entity spec (paaise, SuperBuilder, MembershipStatus restored)
Automated tests	Partly green	mvn test: 19 tests, 18 pass, TenantIsolationIntegrationTest fails to load ApplicationContext
Docker on the dev Mac	Working	docker ps works after starting Docker Desktop
Mobile app on iOS simulator	Running	Login, register, dashboard, role tabs, trainer screens exist
Forest Iron theme and background	Applied	Glass cards, green palette, gym photo behind Home and auth
Trainer module (backend and mobile)	In progress	Screens for notes, trainers list and gym code are visible; access rules need tests
Register screen changes	Partly	City still present, selector not visible in last screenshot
Role-based tabs	Buggy	Owner sees 7 tabs including Notes and Profile; tab bar translucent
Safe area and scroll padding	Buggy	Greeting overlap fixed once; bottom content hidden behind tab bar
Mongo version alignment	Open	Tests used 6.0, production compose uses 7; transactions need replica set
Deployment on Fedora, tunnel, backups	Not started	Server plan documented in the TRD
APK build, pilot	Not started	

3. Roadmap and Milestones

Phase	Weeks (2026-27)	Goal	Exit criteria
0. Stabilise	W1 (12 to 18 Oct)	Green tests, correct role tabs and layout, Register finished	mvn clean test green; three role logins show correct tabs; no screen overlaps
1. Trainer and privacy	W2 to W3 (19 Oct to 1 Nov)	Trainer backend with masking, assignments, notes, approval, tests	Trainer access test suite green; full phone never leaks
2. Trainer UX and owner tools	W4 to W5 (2 to 15 Nov)	Owner trainer management screens, trainer app, audit log, staff toggle	End-to-end demo of trainer join, approve, assign, notes
3. Daily-use polish	W6 to W7 (16 to 29 Nov)	Payments, receipts, check-in speed, QR, reminders, subscription screens, settings	All PRD must-haves pass acceptance criteria
4. Hardening	W8 (30 Nov to 6 Dec)	Security audit, performance review, error handling, mobile review, low-end Android test	No critical or high findings open
5. Deployment	W9 (7 to 13 Dec)	Fedora server, Docker Compose, Cloudflare Tunnel, backups, monitoring	App works over HTTPS from a real phone; restore tested
6. Build and internal test	W10 (14 to 20 Dec)	Android APK, real device testing, privacy policy and terms	Two people use it for a week on their own phones
7. Pilot	W11 to W14 (21 Dec to 17 Jan)	2 to 3 gyms use it daily	Weekly feedback, activation and usage metrics collected
8. Launch v1.0	W15 to W16 (18 to 31 Jan)	Fix pilot issues, set pricing, open for paid signups	First paying gyms; support process working



4. Sprint Plan and Task Backlog

Estimates are in ideal days (one day is about 5 focused hours). Requirement IDs refer to the PRD. Every task is done only when it meets the Definition of Done (section 7).

Sprint 0 and 1 (W1): Stabilise

ID	Task	Ref	Days	Needs
S1-01	Find the root cause of the TenantIsolationIntegrationTest context failure (last "Caused by" in surefire report). Likely JWT_SECRET missing in test config, Mongo URI not wired from Testcontainers, or index conflict. Fix without weakening assertions	TRD 9	0.5	
S1-02	Switch Testcontainers to mongo:7 singleton container shared by all tests; run Mongo as a single-node replica set for transaction tests	TRD 4.2	0.5	S1-01
S1-03	Run mvn clean test until fully green; commit and tag v0.1	-	0.5	S1-02
S1-04	Role-based tabs: tabsByRole config, custom floating tab bar (solid, five tabs for Owner and Staff, four for Trainer), route guards	UI 6.6	1.5	
S1-05	Shared Screen component: safe-area top padding, bottom padding for tab bar, background under status bar and home indicator	UI 4.1	0.5	

ID	Task	Ref	Days	Needs
S1-06	Profile and More clean-up: one user source from /me, Owner profile only in More, gym code visible to Owner only, remove duplicates	AUTH, UI 11	0.5	S1-04
S1-07	Notes screen fixes: header, avatar, single Add Note button, correct empty icon	TRN-07	0.5	
S1-08	Register: remove City, add Owner or Trainer selector, trainer fields and Start button, fix scroll clipping, sage cursor	AUTH-01 to 04	1	
S1-09	Replace logo with a real transparent mint PNG or SVG; app icon and splash	UI 16	0.5	
S1-10	Test with Owner, Staff and Trainer accounts, record the tabs each sees, screenshot all three	-	0.5	S1-04

Sprint 2 (W2): Trainer backend and privacy

ID	Task	Ref	Days	Needs
S2-01	TRAINER role, UserStatus (PENDING_APPROVAL, REJECTED), register-trainer endpoint with gym-code lookup and rate limits	AUTH-04, 05, 09	1	S1-03
S2-02	Gym code generation, uniqueness, regenerate, migration for existing gyms	AUTH-06	0.5	
S2-03	Trainer types collection with defaults and CRUD; approval endpoint requiring types	TRN-01, 02	1	S2-01
S2-04	Assignments collection and endpoints (assign, bulk, end, reassign); deactivation ends assignments	TRN-03, 08	1.5	S2-03
S2-05	Trainer access rules in services: assigned-only lists, search, detail, attendance; 404 for others	TRN-05	1	S2-04
S2-06	Masking: role DTOs for member responses, MaskingUtils, no full phone in QR results, logs, errors; staffCanSeeFullPhone setting	TRN-06, STF-03	1	
S2-07	Member notes API and progress series	TRN-07	0.5	S2-05
S2-08	Audit log collection and endpoint	TRN-09	0.5	

Sprint 3 (W3): Trainer tests and review

ID	Task	Ref	Days	Needs
S3-01	Trainer and masking integration tests (Gym A and Gym B): unassigned 404, raw JSON contains no full phone, 403 on payments, assignment end removes access	TRD 9	1.5	S2-07
S3-02	Security review prompt on trainer code: IDOR, PII in logs, aggregations bypassing assignment filter	TRD 3	1	S3-01
S3-03	Fix findings, rerun full suite, tag v0.2	-	1	S3-02
S3-04	Generate TypeScript types from OpenAPI for the app	TRD 7.2	0.5	S3-03

Sprint 4 and 5 (W4 to W5): Trainer UX and owner tools

ID	Task	Ref	Days
S4-01	Owner: Trainers list with badge count, pending requests, approve sheet with type multi-select, reject	TRN-01	1.5
S4-02	Owner: Trainer types screen; trainer detail with type chips and assigned members	TRN-02	1
S4-03	Owner: Assign members screen (search, multi-select, PT fee), member detail Assigned trainer card	TRN-03	1.5
S4-04	Owner: deactivate trainer and Reassign banner	TRN-08	0.5

ID	Task	Ref	Days
S4-05	Gym settings: gym code (copy, share), allow expired check-in, staff full phone toggle with warning, reminder days	SUB-04	1
S4-06	Audit log screen	TRN-09	0.5
S5-01	Trainer app: My Members, member detail (masked, no payments), lock icon, 404 handling	TRN-05, 06	1.5
S5-02	Trainer notes: add or edit note, filters, weight chart, member chips	TRN-07	1.5
S5-03	Staff behaviour: masked numbers, hide Call and WhatsApp, owner-only controls hidden	STF	0.5
S5-04	Request sent screen and login messages for pending and rejected	AUTH-05	0.5
S5-05	Three-account manual test and screenshots; tag v0.3	-	0.5

Sprint 6 and 7 (W6 to W7): Daily-use polish

ID	Task	Ref	Days
S6-01	Add member 3-step form with live totals, photo compression, duplicate phone error	MEM-01, 02	1.5
S6-02	Member detail: progress ring, payments and attendance tabs, Renew, QR card share	MEM-04, 07, 09	1.5
S6-03	Collect payment: idempotency, success screen, receipt PDF download and WhatsApp share, reversal (Owner)	PAY-01 to 07	2
S6-04	Payments history filters and totals by mode; Dues list with reminders	PAY-05, 06	1
S7-01	Check-in: search and tap, success, already, expired states with haptics; today list with date picker	ATT-01, 03 to 05	1.5
S7-02	QR scanner: continuous mode, cooldown, torch, permissions	ATT-02	1
S7-03	Absent members screen with 3, 7, 15, 30 days	ATT-06	0.5
S7-04	Reminder job, logs, Expiring Soon list with WhatsApp links	REM	1
S7-05	Subscription: banner, screen, expired block, Super Admin list, extend and suspend	SUB	1.5
S7-06	Home polish: count-up, stagger animations, empty state and checklist, expiring list	DASH	1
S7-07	Offline banner, toast system, error boundary, reduce-motion	NFR	1

Sprint 8 (W8): Hardening

ID	Task	Days
S8-01	Full security audit prompt (tenant scoping, IDOR, mass assignment, uploads, JWT, logs, race conditions); fix critical and high	2
S8-02	Performance review: indexes against real queries, pagination, N+1, aggregation cost; seed 100 gyms by 300 members and test	1
S8-03	Mobile review: tokens, error handling, states, FlatList performance, re-renders, role leaks	1
S8-04	Low-end Android test; reduce blur and animations if frames drop	0.5
S8-05	Dependency version pinning and vulnerability scan	0.5
S8-06	Write privacy policy and terms (draft) and in-app consent text; check DPDP Act principles	0.5

Sprint 9 (W9): Deployment

ID	Task	Days
S9-01	Prepare Fedora server: updates, firewalld, Docker, folders, .env (chmod 600)	0.5
S9-02	Compose with MongoDB replica set (auth on, no published port), API on 127.0.0.1:8080, health checks, log rotation	1
S9-03	Domain on Cloudflare, Tunnel as systemd service, HTTPS test from phone data network	1
S9-04	Backup script, cron, rclone to cloud, restore script, restore test into a scratch database	1
S9-05	Uptime Kuma and disk alerts, UPS connected	0.5
S9-06	Smoke test script: register gym, login, add member, pay, check-in, dashboard	0.5

Sprint 10 (W10): Build and internal test

ID	Task	Days
S10-01	EAS Build profiles (preview and production), package id, app name, icon, splash, API URL from env	1
S10-02	Install APK on two or three real Android phones; run the full checklist	1
S10-03	Fix blocking bugs; tag v0.9-rc	2
S10-04	Support setup: WhatsApp business number, UPI details, FAQ, onboarding video or PDF	1

5. AI IDE Prompt Order

Prompts already prepared in the project conversation, mapped to sprints. Always paste the Master Prompt first in a new chat or keep it in the IDE rules file. After each prompt: run, test, review prompt, commit.

Sprint	Prompt	Then
S1	Fix prompt for tests: read the surefire report, find root cause, mongo:7 singleton container	mvn clean test
S1	Fix prompt: role-based tabs, tab bar design, layout (single combined fix prompt with the verification table)	Screenshot Owner, Staff, Trainer
S1	Register restyle prompt (selector, trainer fields, no city)	Screenshot both modes
S2	T-1 Backend roles, trainer types, assignments, masking	Review prompt T-5
S3	T-2 Trainer and tenant isolation tests	mvn clean test, fix, tag
S4 to S5	T-3 Mobile owner side; T-4 Mobile trainer and staff side	Review prompt, three-account test
S6 to S7	Members 7A, Payments 7B, Attendance 7C, QR 7D, Dashboard 7E, More 7F (use the exact response fields from OpenAPI)	One at a time with review
S8	Security audit, performance review, mobile review, pre-release audit prompts	Fix and add tests
S9	D1 Docker, D2 Backup, then manual runbook (section 9)	Smoke test
S10	D3 Android build	Install on phones

5.1 Rules for working with the AI IDE

1. One module per prompt. Paste the Master Prompt first.
2. Ask for a verification table at the end: each requirement, done or not done, file changed. Reject answers without it. (The AI skipped items when prompts were long.)
3. Do not accept "simplified" entities or enums. Compare with the Backend Schema document.
4. Give exact values (hex colours, field names, error codes), not adjectives.
5. Commit after every working step so you can roll back.

6. Run tests yourself; do not trust "tests pass" in a chat reply.
7. When a prompt is too big, split it. Layout and navigation fixes should be separate from feature work.
8. Paste real error output back into the chat, with the last "Caused by" line.

6. Environments and Configuration

Environment	Where	Purpose	Data
Dev	Mac, Docker Desktop, iOS simulator, Android emulator or phone	Daily building	Throwaway data
Test (CI)	Mac or GitHub Actions with Docker for Testcontainers	Automated tests with mongo:7	Ephemeral
Staging (optional)	Same Fedora server with a second compose project on another port	Final checks before releasing to pilot gyms	Copy of anonymised data
Production	Fedora server, then VPS	Pilot and paying gyms	Real data, backed up

Config item	Dev	Production
API URL in app (EXPO_PUBLIC_API_URL)	http://localhost:8080 (simulator) or Mac LAN IP (real phone)	https://api.yourdomain.com
JWT_SECRET	Dev value in local .env	Strong random value, only on the server
MongoDB	Docker with auth, replica set rs0	Same, volume with SELinux label, not exposed
Profiles	dev (Swagger on)	prod (Swagger off or protected)
Logging	Debug	Info, no PII

7. Engineering Practices

7.1 Definition of Done (every feature)

1. Code written and follows the layered architecture and naming conventions.
2. gymId filter present on every query; role check present on every endpoint; trainer or staff masking applied where relevant.
3. Inputs validated; errors map to the standard error codes.
4. Unit and integration tests added; mvn clean test is green.
5. Mobile screen has loading, empty, error and offline states and prevents double submit.
6. Checked on a real phone or simulator with the correct role account.
7. No hard-coded colours or strings (tokens and strings file only).
8. Reviewed with the review prompt; committed with a clear message.

7.2 Git workflow

- Branches: main (stable), develop, feature/<module>. Merge to develop when the module is done and tests pass; merge develop to main at the end of each sprint and tag.
- Commit messages: type(scope): message, for example feat(trainer): add assignment endpoints, fix(ui): safe area on Home.
- Tags: v0.1 after stabilise, v0.2 trainer backend, v0.3 trainer UX, v0.9-rc release candidate, v1.0 launch.
- Never commit .env, keystores, node_modules, target, logs or uploads. Keep an .env.example.
- Make a backup copy of the repository on a second remote (GitHub private plus an external drive).

7.3 Code review checklist (self-review)

- Any findById without gymId?
- Any endpoint missing @PreAuthorize?
- Any response that can include a full phone for STAFF or TRAINER?
- Any money as double?
- Any log line with personal data?
- Any new query without an index?
- Any hard delete?
- Any screen showing data the role should not see?

8. Test Plan and Release Gates

Gate	Checks	Owner
Every commit	Compile, unit tests	Developer
Every module	Integration tests, review prompt, manual test on simulator	Developer
End of sprint	Full mvn clean test (includes tenant isolation and trainer tests), three-account tab check, tag	Developer
Before pilot	Security audit done, backup and restore verified, smoke test on production URL, privacy policy live, APK tested on three phones	Developer
Before launch	Pilot issues closed, pricing and support ready, monitoring alerts tested	Founder

8.1 Manual regression checklist (run before each release)

Area	Check
Auth	Register owner, register trainer, pending login blocked, approve, login works, wrong password message, lock after 5 attempts
Tabs	Owner and Staff see five tabs without Notes or Profile; Trainer sees four tabs; deep link to a forbidden route redirects
Members	Add with photo, duplicate phone error, search, filters, edit, soft delete (Owner only), renew expired and active cases
Payments	Partial payment, overpayment rejected, double tap creates one payment, receipt PDF opens and shares, reversal updates dues
Attendance	Check-in, already checked in, expired warning, QR scan, absent list
Trainer	Sees only assigned, masked phone, no payment screen, unassigned member returns not found, notes and chart work
Staff	Masked phone, no Call or WhatsApp, cannot reverse or delete; toggle on shows full number
Subscription	Banner at 7 days, grace, expired block message, super admin extend restores access
Layout	No overlap with status bar, last card visible above the tab bar, keyboard does not hide the button, offline banner
Performance	Lists scroll smoothly on a low-end Android phone; animations stay smooth

9. Deployment Runbook (Fedora Server)

Run these steps in order. After each step confirm the check before moving on.

#	Step	Command or action	Check
1	Update system	<code>sudo dnf upgrade -y</code>	No errors
2	Install Docker	Add Docker CE repo, install docker-ce, docker-compose-plugin, enable service, add user to docker group, log out and in	<code>docker run hello-world</code>
3	Firewall	<code>sudo firewall-cmd --permanent --add-service=ssh</code> and reload (no other ports needed with the tunnel)	<code>firewall-cmd --list-all</code>
4	Folders	<code>mkdir -p ~/gym-saas/{backend,data/mongo,data/uploads,data/backups}</code>	Folders exist
5	Secrets	Create <code>.env</code> with <code>MONGO_ROOT_USER</code> , <code>MONGO_ROOT_PASSWORD</code> , <code>JWT_SECRET</code> (<code>openssl rand -base64 48</code>); <code>chmod 600 .env</code>	Not in git
6	Code	<code>git clone</code> the backend into <code>~/gym-saas/backend</code> ; copy <code>docker-compose.yml</code>	Files present
7	Start	<code>docker compose up -d --build</code> ; <code>docker compose logs -f api</code>	Health endpoint OK
8	Mongo replica set	Initialise <code>rs0</code> once (<code>mongo-init.js</code>), confirm primary	<code>rs.status()</code> ok
9	Cloudflare Tunnel	Install cloudflared, login, create tunnel, route DNS <code>api.yourdomain.com</code> , config to <code>http://localhost:8080</code> , install as service	<code>https://api...</code> opens Swagger or health
10	Backups	<code>backup.sh</code> with <code>mongodump</code> and <code>rclone</code> ; <code>cron 0 2 * * *</code> ; copy uploads too	File appears next morning
11	Restore test	Restore latest dump into a scratch database	Counts match
12	Monitoring	Uptime Kuma monitor on <code>/actuator/health</code> , alert to WhatsApp or email; disk alert	Test alert received
13	UPS and reboot test	Connect UPS, reboot server, confirm all containers and the tunnel restart on their own	App works after reboot
14	Smoke test	From a phone on mobile data: register, login, add member, pay, check-in, dashboard	All pass

Do not run `mvn test` on the server. Tests run on the Mac (they need Docker Desktop). The server only builds and runs the Docker image.

10. Pilot Plan

10.1 Selecting pilot gyms

- 2 to 3 gyms, 80 to 300 members each, an owner who is willing to give weekly feedback.
- One gym with trainers (to test the trainer module), one small owner-run gym, one with a front-desk staff member.
- Offer free usage during the pilot and a founding-customer discount after.

10.2 Onboarding script (30 minutes per gym)

1. Install the APK and register the gym with the owner on their phone.
2. Create plans together (Monthly, Quarterly, Yearly) with their real prices.
3. Add 5 existing members live; import the rest over the next days (owner or founder helps).
4. Create staff account; explain masked numbers and the setting.
5. Share the gym code with trainers; approve them and assign two or three members.

6. Show Collect fee, receipts, Check-in and Expiring Soon.
7. Agree on a WhatsApp support group and a weekly 15-minute review call.

10.3 What to measure

Metric	Target in pilot
Members entered by day 7	At least 70 percent of active members
Daily check-ins recorded	More than 50 percent of real visits
Payments recorded in app	More than 80 percent of fees collected
Crashes or blocking bugs	Zero open at the end of week 2
Owner satisfaction (1 to 5)	At least 4
Willingness to pay at the end	At least 2 of 3 gyms

10.4 Feedback loop

- Collect feedback in one sheet: date, gym, issue, severity, status.
- Fix severity 1 (data wrong, cannot work) within 24 hours, severity 2 within one week.
- Do not add new features during the pilot unless they block daily use.

11. Subscription Operations (Phase 1, manual)

Event	Action
Owner wants to pay	Share UPI ID and amount (₹499 monthly or ₹4,999 yearly). Owner sends payment screenshot on WhatsApp
Payment received	Super admin extends the gym (monthly or yearly). Send a simple receipt message. Record in a sheet
Reminder to owner	Message at 7 days and 1 day before validity ends
No payment after grace	Access blocks automatically with a clear message; data stays safe; follow up once
Refund or dispute	Policy in Terms: no refund after use of the period; goodwill at founder discretion
Phase 2	Razorpay subscriptions with auto-renew and invoices

12. Budget Estimate (approximate, verify before paying)

Item	Cost (approx.)	When	Notes
Domain name	₹800 to 1,200 per year	Before deployment	Needed for Cloudflare Tunnel and HTTPS
Cloudflare (free plan) and Tunnel	₹0	Deployment	
UPS for the server	₹3,000 to 6,000 one time	Before pilot	Keeps the server up during short power cuts
External drive or cloud backup storage	₹0 to 500 per month	Deployment	Google Drive or Backblaze via rclone
Google Play developer account	About US\$25 one time	Before store release	APK sharing works without it during the pilot
Apple Developer Program	About US\$99 per year	When iOS release is needed	Not needed for Android-first launch

Item	Cost (approx.)	When	Notes
VPS (after 10 or more paying gyms)	₹500 to 800 per month	Phase 2	Docker makes the move easy
WhatsApp Business API or SMS provider	Per message charges	Phase 2	SMS in India needs DLT registration
Expo EAS builds	Free tier for the start	Build	Paid plan only if build queue is too slow
AI coding tool subscription	Your current plan	Ongoing	

13. Risk Register (Delivery)

Risk	Impact	Early warning	Response
AI skips parts of long prompts	Rework, wrong UI	Screenshots differ from the spec	Short prompts, verification table, check against these documents
Tenant leak bug	Critical	Isolation test fails	Stop feature work; fix first; add test
Trainer privacy leak	High	Full phone appears in any response	Server-side DTO fix; add JSON assertion test
Home server outage during pilot	High	Uptime alerts	UPS; communicate; move to VPS if repeated
Backup not restorable	Critical	Restore test fails	Fix script; test weekly until stable
Scope creep	Medium	New feature requests mid-sprint	Write to Phase 2 backlog, not into the sprint
Low-end Android lag	Medium	Dropped frames on cheap phone	Reduce blur and animation; reduce-motion
Solo developer burnout or delays	Medium	Two sprints slip	Cut polish items before cutting security or tests
Unlicensed assets	Low	Brand logos visible	Use own or licensed images; credits

14. Phase 2 and Phase 3 Backlog

Priority	Item	Value
P1	Automatic WhatsApp reminders (Cloud API or provider) with templates	Biggest time saver for owners
P1	Razorpay subscriptions for Gymholik plans, invoices, auto-renew	Removes manual collection
P1	OTP login and forgot password	Security and support load
P2	Reports: monthly income, member growth, churn, trainer-wise PT revenue; export	Owner insight
P2	Class batches and timetable (Zumba, cardio, yoga) with class attendance	Group trainers
P2	PT session counter and trainer commission	Trainer economics
P2	Member app: own QR, attendance, receipts, plan expiry, payments by UPI	Retention
P2	Plan freeze, upgrade, family and couple plans, offers and coupons	Revenue
P3	Multi-branch, chain owner view	Bigger customers
P3	Biometric or face attendance integration	Convenience
P3	Expense tracking, GST invoice, accounting export	Finance
P3	Diet and workout plan builder, body composition charts	Differentiation
P3	Hindi and regional languages	Reach

Priority	Item	Value
P3	Admin web panel for Super Admin	Operations

15. Next Seven Days (Action List)

1. Open the TenantIsolationIntegrationTest report: `grep -E "Caused by" target/surefire-reports/*TenantIsolationIntegrationTest.txt | tail -5`. Fix the root cause (most likely JWT_SECRET missing in test config or Mongo URI not wired).
2. Change Testcontainers to mongo:7 (singleton). Run `mvn clean test` until all 19 tests are green. Commit and tag v0.1.
3. Run the combined fix prompt for role tabs, tab bar and layout. Insist on the verification table in the reply.
4. Log in as Owner, Staff and Trainer accounts. Take a screenshot of the tab bar for each and compare with the UI/UX brief (sections 6.6 and 11).
5. Finish the Register screen: no City, Owner or Trainer selector at the top, trainer gym code and Start button, nothing clipped.
6. Replace the logo with a real transparent mint PNG or SVG; set the sage cursor colour.
7. Review the backend against the Backend Schema document: UserStatus, TRAINER role, trainer collections and indexes; list gaps.
8. Buy the domain and create the Cloudflare account (no rush on the server yet).
9. Pick two or three possible pilot gyms and have a first conversation with each owner; ask what they use today and what hurts most.
10. Create a private Git remote backup and an `.env.example` file if not present.